

Module 18: CASE Statement

The **CASE** statement is SQL's conditional logic operator. It works similarly to IF-ELSE statements in programming languages and allows you to create dynamic outputs, perform conditional calculations, categorize data, and build advanced business reports.



Introduction

Consider an Employees table:

EmployeeName	Salary
Kunal	30000
Rahul	60000
Aman	90000

Management asks:

Categorize employees as Low, Medium, and High salary earners.

Expected Output:

EmployeeName	Salary	SalaryCategory
Kunal	30000	Low
Rahul	60000	Medium
Aman	90000	High

To perform such conditional logic, SQL provides:

CASE



Learning Objectives

After completing this module, you will be able to:

- ✓ Understand CASE statements
- ✓ Use Simple CASE
- ✓ Use Searched CASE

- ✓ Use CASE in SELECT
 - ✓ Use CASE in ORDER BY
 - ✓ Use CASE with Aggregates
 - ✓ Perform Conditional Aggregation
 - ✓ Build Dynamic Reports
-



Table of Contents

1. What is CASE?
 2. Why CASE is Needed
 3. Types of CASE
 4. Simple CASE
 5. Searched CASE
 6. CASE in SELECT
 7. CASE in ORDER BY
 8. CASE with Aggregate Functions
 9. Conditional Aggregation
 10. Nested CASE
 11. Common Mistakes
 12. Best Practices
 13. Interview Questions
 14. Business Analytics Examples
 15. Summary
 16. Practice Questions
-



What is CASE?

The CASE statement adds conditional logic to SQL queries.

Similar To

Programming:

```
IF  
ELSE IF  
ELSE
```

SQL:

```
CASE
  WHEN Condition
  THEN Result

  ELSE Result
END
```

Basic Example

```
SELECT EmployeeName,

  CASE
    WHEN Salary > 50000
    THEN 'High Salary'

    ELSE 'Low Salary'
  END AS SalaryStatus

FROM Employees;
```

Why CASE Matters

Used in:

- Dashboards
- Business Reports
- KPI Calculations
- Data Categorization
- Analytics Projects

2 Why CASE is Needed

Without CASE:

```
Raw Data Only
```

With CASE:

Business Meaning

Example:

Raw Salary:

45000

Business Category:

Medium Salary

CASE converts data into understandable business information.

3 Types of CASE

SQL provides two forms:

Type	Purpose
Simple CASE	Compare exact values
Searched CASE	Evaluate conditions

Simple CASE

```
CASE ColumnName
  WHEN Value1 THEN Result1
  WHEN Value2 THEN Result2
END
```

Searched CASE

```
CASE
  WHEN Condition1 THEN Result1
  WHEN Condition2 THEN Result2
END
```

Simple CASE

Used when comparing exact values.

Example Table

DepartmentID
1
2
3

Query

```
SELECT DepartmentID,
       CASE DepartmentID
         WHEN 1 THEN 'IT'
         WHEN 2 THEN 'HR'
         WHEN 3 THEN 'Finance'
         ELSE 'Unknown'
       END AS DepartmentName
FROM Employees;
```

Output

DepartmentID	DepartmentName
1	IT
2	HR

DepartmentID	DepartmentName
3	Finance

Best Use Case

Mapping codes to labels.

Examples:

Status Codes
Country Codes
Department IDs
Grades

5 Searched CASE

Most commonly used form.

Allows logical conditions.

Example

```
SELECT EmployeeName,
       Salary,

       CASE

           WHEN Salary < 40000
               THEN 'Low'

           WHEN Salary BETWEEN 40000 AND 70000
               THEN 'Medium'

           ELSE 'High'

       END AS SalaryCategory

FROM Employees;
```

Output

EmployeeName	Salary	SalaryCategory
Kunal	30000	Low
Rahul	60000	Medium
Aman	90000	High

Execution Rule

CASE executes from top to bottom.

First matching condition wins.

Example

```
CASE  
  
WHEN Salary > 20000 THEN 'A'  
  
WHEN Salary > 50000 THEN 'B'  
  
END
```

Salary:

```
60000
```

Result:

```
A
```

because first condition matched.

Always Order Conditions Carefully



CASE in SELECT

Most common use.

Example

```
SELECT EmployeeName,  
  
       CASE  
         WHEN Salary > 50000  
         THEN 'Eligible'  
  
         ELSE 'Not Eligible'  
       END AS BonusStatus  
  
FROM Employees;
```

Output

EmployeeName	BonusStatus
Kunal	No
Rahul	Yes

Business Uses

- Salary Bands
- Risk Levels
- Customer Segments
- Performance Ratings

7 CASE in ORDER BY

Dynamic sorting.

Example

```
SELECT EmployeeName,  
       Department  
  
FROM Employees  
  
ORDER BY
```



```
CASE Department

  WHEN 'IT' THEN 1

  WHEN 'HR' THEN 2

  WHEN 'Finance' THEN 3

END;
```

Result:

```
IT First
HR Second
Finance Third
```

Why?

Custom sort order.

8 CASE with Aggregate Functions

CASE can work with aggregates.

Example

Count employees with high salaries.

```
SELECT COUNT(

CASE

  WHEN Salary > 50000

  THEN 1

END

) AS HighSalaryEmployees

FROM Employees;
```

Alternative

```
SELECT SUM(  
  
CASE  
  
    WHEN Salary > 50000  
  
    THEN 1  
  
    ELSE 0  
  
END  
  
) AS HighSalaryEmployees  
  
FROM Employees;
```



Conditional Aggregation

One of the most important Data Analytics techniques.

Example

```
SELECT  
  
SUM(  
CASE  
    WHEN Department='IT'  
    THEN Salary  
    ELSE 0  
END  
) AS ITSalary,  
  
SUM(  
CASE  
    WHEN Department='HR'  
    THEN Salary  
    ELSE 0  
END  
) AS HRSalary  
  
FROM Employees;
```

Output

ITSalary	HRSalary
150000	120000

Why Important?

Used in:

- KPI Dashboards
 - Power BI
 - Tableau
 - Financial Reports
-

Example

Employee Count by Department

```
SELECT

SUM(
CASE
WHEN Department='IT'
THEN 1
ELSE 0
END
) AS ITEmployees,

SUM(
CASE
WHEN Department='HR'
THEN 1
ELSE 0
END
) AS HREmployees

FROM Employees;
```

10 Nested CASE

CASE inside CASE.

Example

```
SELECT EmployeeName,  
  
CASE  
  
    WHEN Department='IT'  
  
        THEN  
  
            CASE  
  
                WHEN Salary > 70000  
                THEN 'Senior IT'  
  
                ELSE 'Junior IT'  
  
            END  
  
        ELSE 'Other Department'  
  
END AS EmployeeCategory  
  
FROM Employees;
```

Use Carefully

Can reduce readability.

1**1**

Common Mistakes

Missing END

Bad:

```
CASE  
WHEN Salary > 50000  
THEN 'High'
```

Error.

Correct:

```
CASE
WHEN Salary > 50000
THEN 'High'
END
```

Wrong Condition Order

Bad:

```
WHEN Salary > 20000
WHEN Salary > 50000
```

More specific conditions should appear first.

Missing ELSE

Allowed but risky.

Example

```
CASE
WHEN Salary > 50000
THEN 'High'
END
```

Unmatched rows become:

```
NULL
```

Mixing Data Types

Bad:

```
CASE  
  
WHEN Salary > 50000  
  
THEN 'High'  
  
ELSE 100  
  
END
```

Can cause conversion issues.

1 2 Best Practices

Always Include ELSE

Good:

```
ELSE 'Unknown'
```

Use Meaningful Aliases

Good:

```
SalaryCategory
```

Keep Conditions Ordered

Most specific first.

Avoid Deep Nesting

Hard to maintain.

Use Conditional Aggregation

Very common in analytics.

Professional Example

```
SELECT EmployeeName,  
  
       Salary,  
  
       CASE  
  
           WHEN Salary < 40000  
            THEN 'Low'  
  
           WHEN Salary BETWEEN 40000 AND 70000  
            THEN 'Medium'  
  
           ELSE 'High'  
  
       END AS SalaryBand  
  
FROM Employees;
```

1 3 Common Interview Questions

What is CASE?

Conditional logic in SQL.

Difference Between Simple and Searched CASE?

Simple:

```
CASE Column
```

Searched:

```
CASE  
WHEN Condition
```

Can CASE Be Used in ORDER BY?

Yes.

Can CASE Be Used with Aggregates?

Yes.

What is Conditional Aggregation?

Using CASE inside aggregate functions.

1**4**

Business Analytics Examples

Salary Categorization

```
SELECT EmployeeName,  
  
CASE  
  
WHEN Salary < 40000 THEN 'Low'  
  
WHEN Salary < 70000 THEN 'Medium'  
  
ELSE 'High'  
  
END  
  
FROM Employees;
```

Customer Segmentation

```
SELECT CustomerName,  
  
CASE  
  
WHEN TotalPurchase > 100000  
THEN 'Premium'  
  
WHEN TotalPurchase > 50000  
THEN 'Gold'  
  
ELSE 'Regular'
```



```
END

FROM Customers;
```

Department Salary Dashboard

```
SELECT

SUM(
CASE
WHEN Department='IT'
THEN Salary
ELSE 0
END
) AS ITSalary,

SUM(
CASE
WHEN Department='HR'
THEN Salary
ELSE 0
END
) AS HRSalary

FROM Employees;
```

Employee Performance Rating

```
SELECT EmployeeName,

CASE

WHEN Rating >= 9 THEN 'Excellent'

WHEN Rating >= 7 THEN 'Good'

WHEN Rating >= 5 THEN 'Average'

ELSE 'Needs Improvement'

END
```

```
FROM Employees;
```



Summary

In this module, you learned:

- ✓ Simple CASE
 - ✓ Searched CASE
 - ✓ CASE in SELECT
 - ✓ CASE in ORDER BY
 - ✓ CASE with Aggregates
 - ✓ Conditional Aggregation
 - ✓ Nested CASE
 - ✓ Dashboard Reporting Techniques
-



Practice Questions

Theory

1. What is CASE?
 2. Why is CASE used?
 3. Difference between Simple and Searched CASE?
 4. Can CASE be used in SELECT?
 5. Can CASE be used in ORDER BY?
 6. What is Conditional Aggregation?
 7. Why is ELSE important?
 8. How does CASE execute conditions?
 9. What is Nested CASE?
 10. What are common CASE mistakes?
-

Practical Exercises

Task 1

Categorize salaries:

Low
Medium
High

Task 2

Convert Department IDs into names.

Task 3

Create bonus eligibility status.

Task 4

Use CASE in ORDER BY.

Task 5

Count employees by department using CASE.

Task 6

Calculate department-wise salary totals using CASE.

Challenge Project

Create:

Employees
Departments
Customers
Orders

Generate:

- Salary Bands
- Customer Segments
- Performance Ratings
- Department KPIs
- Dashboard Metrics

using:

- CASE
 - Aggregates
 - Conditional Aggregation
-



Next Module

→ **Module 19: Views**

Topics Covered:

- What are Views?
- Creating Views
- Updating Views
- Dropping Views
- Advantages of Views
- Security Benefits
- View Limitations
- Real-World Reporting Use Cases